Euclidian Algorithm for finding GCD

It is a method to find the GCD of 2 integers. It reduces the problem of finding the GCD of 2 nos. into smaller and more manageable tasks, and with every iteration, the algorithm brings closer to the ~~algori~~ solution.

What is GCD or Highest Common Factor (HCF) ?

It is the greatest common factor that divides each no. such that the remainder is 0.

Example :-

(1) $36 = 2 \times 2 \times 3 \times 3$

$60 = 2 \times 2 \times 3 \times 5$

$(GCD/HCF) = 2 \times 2 \times 3 = 12$

(2) GCD (10, 7) = 1

(3) GCD (10, 5) = 5

Euclidean Algo :-

→ oldest method.

→ based on the principle that GCD of 2 numbers doesn't change if the larger no. is replaced by its difference with the smaller no.

Ex :- GCD (252, 105) = GCD (252-105, 105) = GCD (147, 105) = 21

Steps to find GCD using Euclidean Algorithm.

Let 'a' and 'b' be two numbers such that a > b.

S1 Divide the larger no. 'a' by the smaller no. 'b'.

S2 Replace 'a' with 'b' and 'b' with the remainder from step 1.

S3 Repeat step1 and step2 until the remainder is 0.

S4 Once you get the remainder 0, the divisor will be the GCD of two numbers.

Ex. Find GCD of 48 and 18 using Euclidean Algorithm.

S1 divide 48 by 18, (remainder 12) GCD(48, 18)

S2 Replace 48 by 18 and 18 by 12. GCD(18, 12)

S3. Repeat step 1 for GCD (18, 12) = remainder (6)

S4 Repeat step 2 ⇒ (12, 6)

S5 Repeat step 1 = remainder = 0.

S6 Hence GCD = 6

Chinese Remainder Theorem

Chinese Remainder Theorem states that there always exists an "x" that satisfies the given congruence :-

$$x \equiv rem[0] \pmod{num[0]}$$
$$x \equiv rem[1] \pmod{num[1]}$$

and num[0], num[1] should be co-prime (i.e. they should have GCD/HCF = 1) only then we can find "x".

Ex :- $x = 1 \bmod 5$, $x = 3 \bmod 7$ } Here 5 and 7 should be coprime

It is used to solve various mathematical problem.

$$x \equiv a_1 \pmod{m_1}$$
$$x = a_2 \pmod{m_2}$$
$$x = a_3 \pmod{m_3}$$

Here,

(i) $\gcd(m_1, m_2) = \gcd(m_2, m_3) = \gcd(m_3, m_1) = 1$

(ii) $x = (M_1 X_1 a_1 + M_2 X_2 a_2 + M_3 X_3 a_3 \ldots\ldots + M_n X_n a_n)$

Here M is given by $M = m_1 * m_2 * m_3$.

& $M_i = \frac{M}{m_i}$ eg :- $M_1 = \frac{M}{m_1} = m_2 * m_3$.

So $M_1 = m_2 * m_3$
$M_2 = m_1 * m_3$
$M_3 = m_2 * m_1$

To calculate X_i :-

$$M_i X_i \equiv 1 \bmod m_i$$

eg. $M_1 X_1 \equiv 1 \bmod m_1$

Ex :- $x \equiv 1 \pmod 5 \Rightarrow a_1 = 1, m_1 = 5$
$x \equiv 1 \pmod 7 \Rightarrow a_2 = 1, m_2 = 7$
$x \equiv 3 \pmod{11} \Rightarrow a_3 = 3, m_3 = 11$

Solution :-

Now, $\gcd(5,7) = \gcd(7,11) = \gcd(11,5) = 1$

First, we'll calculate the value of M,

So, $M = m_1 * m_2 * m_3 \Rightarrow 5 \times 7 \times 11 = 385$

Now, we'll calculate the value of M_i,

$$M_1 = \frac{M}{m_1} = m_2 * m_3 = 77$$

$$M_2 = \frac{M}{m_2} = m_1 * m_3 = 55$$

$$M_3 = \frac{M}{m_3} = m_1 * m_2 = 35$$

Now, we'll calculate value of x_i (multiplicative inverse of m_i)

$$M_i x_i \equiv 1 \pmod{m_i}$$

$$\Rightarrow 77 x_1 \equiv 1 \pmod 5$$

dividing 77 by 5 we should get remainder 1.

$$\Rightarrow 2 x_1 \pmod 5 = 1.$$

$$\therefore x_1 = 3$$

Similarly,

$$M_2 x_2 \equiv 1 \pmod{m_2}$$

$$55 x_2 \bmod 7 = 1$$

$$6 x_2 \pmod 7 = 1$$

$$x_2 = 6$$

$$M_3 x_3 \equiv 1 \pmod{m_3}$$

$$35 x_3 \pmod{m_3} = 1$$

$$x_3 = 6.$$

final x value =

$$x = (M_1 x_1 a_1 + M_2 x_2 a_2 + M_3 x_3 a_3 \text{---})$$

$$= (77(3)(1) + 55(6)(1) + 35(6)(3)) \bmod 385$$

$$= 36$$

$36 \bmod 5 = 1$
$36 \bmod 7 = 1$
$36 \bmod 11 = 3$

Modular Arithmatic

When we divide 2 integers, we'll have an equation like this,

$$\frac{A}{B} = Q \text{ remainder } R$$

A = dividend, B = divisor, Q = quotient, R = remainder

But sometimes, we only need the remainder. For this reason, we use an operator called modulo. Now, using the same integer, we get:-

$$A \bmod B = R$$

There is a quotient remainder theorem, which states that for any 2 integers a and b, there exists two unique integers q and r such that

$$a = b \times q + r, \text{ where } 0 \leq r < b$$

Example $a = 17$, $b = 4$ then $q = 4$ and $r = 1$

$$17 = 4 \times 4 + 1$$

Modular Addition

The rule :- $(a+b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$.

$(18 + 17) \bmod 3$

$\Rightarrow ((13 \bmod 3) + (17 \bmod 3)) \bmod 3$

$\Rightarrow (1 + 2) \bmod 3$

$\Rightarrow 3 \bmod 3$

$\Rightarrow 0.$

2.) Modular Subtraction

$$(a-b) \bmod m = ((a \bmod m) - (b \bmod m)) \bmod m.$$

3.) Modular Multiplication

$$(a \times b) \bmod m = ((a \bmod m) \times (b \bmod m)) \bmod m$$

Ex: $(12 \times 13) \bmod 5$

$\Rightarrow ((12 \bmod 5) \times (13 \bmod 5)) \bmod 5$

4.) Modular Division

It is different from other operations. And sometimes it doesn't exist also.

$$(a/b) \bmod m = (a \times (\text{inverse of } b \text{ if exists})) \bmod m.$$

The task here is to find a no. "c" such that $(b * c) \bmod m = a \bmod m$

5.) Modular Inverse

Modular inverse of "a mod m" exists only if "a" and "m" are relatively prime i.e gcd (a, m) = 1. Hence for finding the inverse of "a" under modular m, if $(a \times b) \bmod m = 1$, then b is the modular inverse of a.

Ex :- a = 5, m = 7, $(5 \times 3) \bmod 7 = 1$

hence 3 is modulo inverse of 5 under 7.

Ex: of division:-

① a = 8, b = 4, m = 5

$8 \bmod 5 = (4 * c) \bmod 5$

$3 = (4 \times c) \bmod 5$

here c = 2.

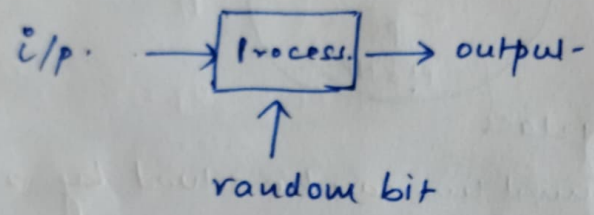
② a = 11, b = 4, m = 5

$11 \bmod 5 = (4 \times c) \bmod 5$

$1 = (4 \times c) \bmod 5 \Rightarrow c = 4$

Randomized Algorithms

Randomized algorithms are the category of algorithms that are not "time-deterministic" in nature. Their correctness is based on the multiple execution of the same algorithm. However, the execution of algorithm is based on the randomness of the input, supplied at the time of execution.



Randomized algorithms are of two types:-

1.) Las Vegas :- algorithm which are determined to give correct output but here the time constraint is variable. Ex:- Randomized Quick Sort, Randomized String Matching in which if the error occurs, Las Vegas will start from the beginning.

2.) Monte-Carlo :- algorithm which is kind of deterministic in context of time but we can't guarantee the correctness of the algorithm. Ex:- If we apply Monte Carlo over string matching algorithm and if there occurs any kind of error, then the algo will resume from the same point next time instead of scanning from the beginning.

US-13

Algorithm :- (Vertex Cover)

G is a graph with vertices V and E.
C is vertex cover.

Step 1 :- C = ϕ // Initialize
Step 2 :- Until there is no edge left in E, repeat steps 3 to 5.
Step 3 :- Pick an edge (u, v) arbitrarily.
Step 4 :- Remove all edges from E of the form (u, w) or (v, w)
Step 5 :- C = C ∪ {u, v}
Step 6 :- Return C.

C-182

Algorithm (Set-Cover) (S, Sn)
// S is the set to be covered
Step 1 :- C = ϕ // Initialize
Step 2 :- U = S
Step 3 :- Until U is empty, repeat step 4 to 6.
Step 4 :- Pick set Si with max |Si ∩ U|
Step 5 :- U = U - Si
Step 6 :- C = C ∪ {Si}.
Step 7 :- Return C.

Example :- Compute set cover for given instance.
S = {a, b, c, d, e, f, g, h}.
S_1 = {a, b, c}
S_2 = {b, d, f, g}
S_3 = {a, e, f, g}
S_4 = {c, d, e, h}
S_5 = {a, h}
S_6 = {b, c, f, h}

NP-complete

Although any given solution to an NP-complete can be verified quickly but there is no known efficient way to locate the solution in first place. The most notable characteristics of NP-complete problems is that no fast solution to them is known. That is, the time required to solve the problems using any currently known algorithm increases very quickly as the size of the problem grows. Ex:-

Approximation algorithms.

In optimization problem, we maximize or minimize an objective function. The maximum or minimum value is called optimum solution of the problem. Some optimization problems are NP-complete problems hence it is difficult to find an optimum solution.

In such cases, we can compute an approximate solution instead of optimum solution.

Ex:-

1) for TSP problem, the optimal solution is the shortest cycle while approximate solution is short cycle.
2) for Knapsack problem, solution is maximum profit while approximate solution is large profit.

each optimization problem can have a correspondi approximate solution.

Vertex Cover

A vertex cover is a set of vertices which covers all the edges of the graph.

Approach :-

- Pick any arbitrary ~~edg~~ vertex and delete all the edges connected to it, as they have been now covered.

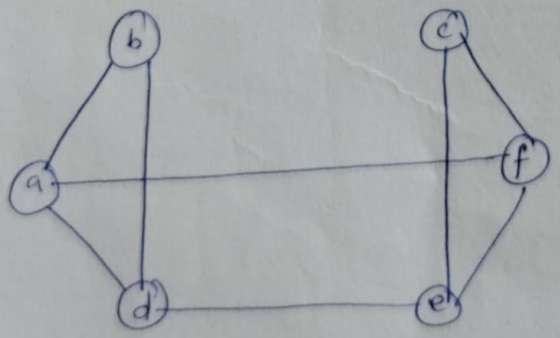
The resulting graph is new instance for the vertex-cover, and the algorithm can be repeated till all edges are covered.

Assignment

write algo for Vertex Cover.

- Question :-

find Vertex Cover for the given graph :-



1) Initialize cover $C = \phi$
2) $E = \{ab, ad, bd, de, af, cf, ef, ce\}$
3) Pick bd arbitrarily.

Remove edges associated with b or d, that is a
and de.

Now $E = \{af, cf, ef, cc\}$.

Pick cf at random.

Remove edges associated with c or f, that is af, cf,

Now $E = \phi$

So, stop.

The cover is $C = \{b, d, c, f\}$.